Архитектурное описание интерпретатора командной строки

Конкретные сигнатуры некоторых методов — задание для разработчика (чучуть не успели)

1. Компоненты системы

1.1 Shell

Контейнер верхнего уровня. Отвечает за сборку системы, хранение глобального состояния между итерациями REPL, организацию цикла чтения-выполнения и координацию работы всех компонентов.

App

- **Назначение.** Точка входа в приложение. Инициализирует и запускает интерпретатор командной строки.

Поля и методы:

- `+ main()` — создаёт экземпляр REPL/интерпретатора, запускает основной цикл работы и передаёт управление на уровень REPL.

Repl (REPL-цикл)

- **Назначение.** Реализует цикл Read–Eval–Print (REPL). Читает строки ввода, передаёт их на обработку координатору, обрабатывает завершение работы (EOF) и продолжает цикл до получения сигнала выхода.

Поля:

- `- interpr : Interpreter` — координатор обработки команд. Используется для выполнения строк ввода (`ExecuteLine`) и проверки флага завершения работы (`ShouldExit`).
- `- prompt : String` — строка приглашения, выводимая перед чтением очередной строки ввода.
- `- stdin : Reader` — поток ввода, из которого считываются строки пользовательского ввода.
- `- stdout : Writer` — поток вывода, в который выводится приглашение (`prompt`).
- `- stderr : Writer` — поток вывода ошибок, используемый для сообщений об ошибках ввода/чтения.

Методы:

- `+ Run() : error` — основной цикл REPL. При каждой итерации проверяет `interpr.ShouldExit()`: если истина — завершает цикл. Вызывает `readline()` для чтения строки ввода. При получении `EOF` завершает работу без ошибки. При ошибке чтения выводит сообщение в `stderr` и переходит к следующей итерации. При успешном чтении строки передаёт её в `interpr.ExecuteLine(line)` и сохраняет полученный код возврата в состоянии интерпретатора (`lastStatus`), после чего повторяет цикл.
- `- readline() : (String, error)` — читает одну строку ввода из `stdin`. Выводит `prompt` в `stdout` перед чтением. Возвращает прочитанную строку (без символа перевода строки) либо ошибку (в том числе `EOF` при завершении ввода).

Interpreter (координатор конвейера обработки строки)

- **Назначение.** Центральный координатор обработки одной строки ввода. Собирает конвейер «лексический анализ → синтаксический разбор → исполнение», хранит глобальное состояние сессии (окружение, реестр команд, флаги завершения и коды возврата), предоставляет точку регистрации встроенных и пользовательских команд.

Поля:

- `- env : Environment` — глобальное окружение интерпретатора. Сохраняется между итерациями REPL. Используется при формировании контекста исполнения по умолчанию (`defaultContext()`), а также при регистрации и выполнении команд.
- `- registry : Registry` — реестр доступных команд. Хранит соответствие имени команды её реализации. Используется для поиска команды при исполнении и для регистрации команд (`registerDefaults()`, `Register()`).
- `- lexer : Lexer` (интерфейс) — абстракция токенизатора. Используется методом `ExecuteLine()` для преобразования входной строки в поток токенов.

- `- parser : Parser` (интерфейс) — абстракция синтаксического анализатора. Используется методом `ExecuteLine()` для построения AST по списку токенов.
- `- expander : Expander` (интерфейс) — абстракция сервиса раскрытия переменных. Передаётся в `Executor` (через композицию/использование) и используется при необходимости раскрытия подстановок в процессе исполнения.
- `- executor : Executor` (интерфейс) — абстракция исполнителя. Используется методом `ExecuteLine()` для обхода и выполнения построенного AST.
- `- stdin : Reader` — поток ввода, закреплённый за текущей сессией REPL. Передаётся в контекст исполнения по умолчанию.
- `- stdout : Writer` — поток вывода текущей сессии REPL. Передаётся в контекст исполнения по умолчанию.
- `- stderr : Writer` — поток вывода ошибок текущей сессии REPL. Используется для вывода сообщений об ошибках лексического и синтаксического анализа в `ExecuteLine()`.
- `- shouldExit : Boolean` — флаг необходимости завершения работы интерпретатора. Устанавливается командами, инициирующими выход (например, `exit`). Проверяется в `Repl.Run()` и доступен через `ShouldExit()`.
- `- exitCode : Integer` — код завершения, который должен быть возвращён при выходе из интерпретатора. Хранит значение, заданное командой завершения.
- `- lastStatus : Integer` — код возврата последней выполненной команды. Сохраняется после каждого вызова `ExecuteLine()` и остаётся доступным в рамках сессии.

Методы:

- `+ Register(c : Command)` — регистрирует команду `c` в `registry`. Если команда с таким именем уже существует — она замещается.
- `+ ExecuteLine(line : String) : Integer` — выполняет обработку одной строки ввода. Последовательно взаимодействует с конвейером: вызывает `lexer.Tokenize()` для получения списка токенов. При ошибке токенизации выводит сообщение об ошибке лексера в `stderr` и возвращает код возврата `1`. Вызывает `parser.Parse()` для построения AST по токенам. При ошибке разбора выводит сообщение об ошибке парсера в `stderr` и возвращает код возврата `1`. Если AST равен `nil` (пустая строка ввода) — возвращает `0`, не выполняя команд. В противном случае формирует контекст исполнения через `defaultContext()` и передаёт AST в `executor.Execute(this, ctx, node)` для исполнения. Возвращает код завершения, полученный от исполнителя. Сохраняет этот код в `lastStatus`.
- `+ ShouldExit() : Boolean` — возвращает значение флага `shouldExit` (истина, если требуется завершить работу интерпретатора).
- `- defaultContext() : ExecContext` — создаёт контекст исполнения по умолчанию для текущей строки: заполняет потоки `stdin/stdout/stderr` значениями из полей интерпретатора и устанавливает `env` в текущее глобальное окружение (`env`). Используется `ExecuteLine()` перед передачей AST в `Executor`.
- `- registerDefaults()` — регистрирует стандартный набор встроенных команд в `registry`. Вызывается при инициализации системы для наполнения реестра командами по умолчанию.

Registry

- **Назначение.** Хранит отображение имён команд на их реализации. Обеспечивает поиск, регистрацию и перечисление доступных команд.

Поля:

- `- cmds : Dict(String, Command)` — ассоциативный массив имён команд и соответствующих экземпляров, реализующих интерфейс `Command`.

Методы:

- `+ Get(name : String) : (Command, Boolean)` — ищет команду по имени `name` в `cmds`. Возвращает пару `(command, found)`, где `found` — признак наличия команды в реестре. Если команда не найдена, `found` равен `false`.
- `+ Register(c : Command)` — регистрирует команду `c`. Ключом регистрации выступает `c.Name()`.
- `+ Names() : String [*]` — формирует список имён всех зарегистрированных команд (в произвольном порядке обхода словаря). Возвращает массив строк с именами команд.

Environment

- **Назначение.** Представляет окружение переменных shell. Хранит значения переменных и признак их экспортируемости во внешние процессы. Сохраняет состояние между итерациями REPL, поддерживает клонирование для создания локального окружения команды.

Поля:

- `- vars : Dict(String, String)` — словарь переменных окружения: ключ — имя переменной (`String`), значение — её строковое значение (`String`). Содержит все переменные, присутствующие в окружении.
- `- exported : Dict(String, Boolean)` — словарь флагов экспортируемости переменных: ключ — имя переменной, значение `Boolean` — признак того, что переменная должна передаваться во внешние процессы при запуске. Используется для формирования списка окружения внешней команды (`Environ()`).

Методы:

- `+ FromOS() : Environment` — создаёт и инициализирует экземпляр `Environment` на основе переменных операционной системы. Заполняет `vars` парами `NAME=VALUE` из системного окружения, при этом все импортированные переменные помечаются как экспортируемые (устанавливаются соответствующие флаги в `exported`). Возвращает созданный экземпляр.
- `+ Get(name : String) : (String, Boolean)` — возвращает значение переменной `name`. Возвращает пару `(value, exists)`, где `exists` равен `true`, если переменная определена в `vars`. Если переменная отсутствует — `exists` равен `false`, `value` не определён.
- `+ Set(name : String, value : String)` — устанавливает значение переменной `name` равным `value`. Записывает пару в `vars`. Флаг экспортируемости `exported[name]` при этом не изменяется (локально заданные переменные по умолчанию не являются экспортируемыми).
- `+ Environ() : String [*]` — формирует список переменных окружения в формате `NAME=VALUE` для передачи во внешние процессы. Включает только те переменные, для которых в `exported` установлен признак `true`. Возвращает массив строк соответствующего формата.
- `+ Clone() : Environment` — создаёт глубокую копию текущего окружения. Копирует содержимое `vars` и `exported` в новые словари. Возвращает независимый экземпляр `Environment`, изменения в котором не влияют на исходный.
- `- markExported(name : String)` — помечает переменную `name` как экспортируемую. Устанавливает `exported[name] = true`. Используется при необходимости временно сделать переменную доступной для дочернего процесса (в рамках локального окружения конкретной команды).

Reader (интерфейс)

- **Назначение.** Абстракция потока ввода. Определяет границу с внешним миром для операций чтения.
- `+ Read(p : Byte [*]) : (Integer, error)` — читает байты из потока ввода в буфер `p`. Возвращает количество прочитанных байтов (`Integer`) и `error` (включая `EOF` при завершении потока).

Writer (интерфейс)

- **Назначение.** Абстракция потока вывода. Определяет границу с внешним миром для операций записи.
- `+ Write(p : Byte [*]) : (Integer, error)` — записывает байты из буфера `p` в поток вывода. Возвращает количество записанных байтов (`Integer`) и `error` в случае неудачи.

1.2 Lexer

Компонент лексического анализа. Преобразует входную строку в поток токенов, сохраняя информацию о кавычках и структуре слов. Не производит раскрытия переменных и не анализирует синтаксическую структуру команд.

Lexer (реализация)

- **Назначение.** Выполняет токенизацию входной строки. Разбивает строку на токены `Word`, `Pipe`, `EOF`, формирует слова (`Word`) из сегментов (`WordPart`) с учётом одинарных и двойных кавычек.

Поля:

- `- input : String` — исходная строка ввода, подлежащая токенизации.

- `- pos : Integer` — текущая позиция сканирования в `input` (индекс символа).

Методы:

- `+ Tokenize() : (Token [*], error)` — запускает процесс токенизации всей входной строки `input`. Последовательно пропускает пробелы, выделяет слова (`scanWord()`), распознаёт символ `|` как токен `Pipe`, по достижении конца строки формирует токен `EOF`. Собирает последовательность токенов и возвращает её. В случае ошибки разбора (например, незакрытая кавычка) возвращает `error`.
- `- skipSpaces()` — пропускает пробельные символы в текущей позиции `pos`, сдвигая указатель сканирования вперёд до первого непробельного символа.
- `- scanWord() : (Word, error)` — извлекает одно слово из входной строки, начиная с текущей позиции. Учитывает кавычки: при обнаружении `'` вызывает `scanSingleQuoted()`, при обнаружении `"` вызывает `scanDoubleQuoted()`, в противном случае накапливает нецитированный текст и при необходимости вызывает `flushUnquoted()` для формирования сегмента `WordPart`. Формирует объект `Word`, состоящий из набора `WordPart`. Возвращает слово или `error` при ошибке чтения внутри кавычек.
- `- scanSingleQuoted() : (WordPart, error)` — сканирует содержимое, заключённое в одинарные кавычки. Считывает символы до закрывающей одинарной кавычки, не интерпретируя внутри символ `$` как начало подстановки. Формирует `WordPart` с `quoteKind = Single`, `expandable = false`, `text` — содержимым внутри кавычек. При отсутствии закрывающей кавычки возвращает `error`.
- `- scanDoubleQuoted() : (WordPart, error)` — сканирует содержимое, заключённое в двойные кавычки. Считывает символы до закрывающей двойной кавычки. Содержимое внутри двойных кавычек может содержать подстановки переменных (`$`), поэтому для соответствующего сегмента устанавливается `expandable = true`, `quoteKind = Double`. Формирует `WordPart` с текстом содержимого. При отсутствии закрывающей кавычки возвращает `error`.
- `- flushUnquoted(parts : WordPart [*], buf : String)` — преобразует накопленный буфер нецитированного текста `buf` в `WordPart` с `quoteKind = Unquoted`, `expandable = true` (нецитированный текст допускает раскрытие переменных). Добавляет полученный сегмент в список `parts`. Очищает буфер после добавления.

Token

- **Назначение.** Элемент потока токенов, получаемого после лексического анализа.

Поля:

- `- type : TokenType` — тип токена (`Word`, `Pipe`, `EOF`).
- `- word : Word` — слово, связанное с токеном. Заполняется для токенов типа `Word`, для остальных может быть пустым.
- `- raw : String` — исходное текстовое представление токена (исходная подстрока строки ввода, соответствующая данному токену).
- `- position : Integer` — позиция в исходной строке `input`, с которой начинается данный токен.

TokenType (перечисление)

- **Назначение.** Классифицирует типы токенов, порождаемых лексером.
- `Word` — токен, представляющий слово (команду, аргумент или присваивание). Содержит `Word`.
- `Pipe` — токен символа `|` (разделитель команд в пайплайне).
- `EOF` — токен конца входной строки.

1.3 Parser

Компонент синтаксического анализа. Преобразует поток токенов в абстрактное синтаксическое дерево (AST), отражающее структуру команды или пайплайна. Не раскрывает переменные и не опирается на окружение.

Parser (реализация)

- **Назначение.** Строит AST по списку токенов в соответствии с грамматикой: `pipeline := simpleCommand ( '|' simpleCommand )*`, `simpleCommand := assignment* word*`.

Поля:

- `- tokens : Token [*]` — список токенов, подлежащих разбору.
- `- pos : Integer` — текущий индекс токена в списке `tokens`.

Методы:

- `+ Parse() : (Node, error)` — точка входа синтаксического анализа. Если текущий токен — `EOF`, возвращает `nil` (пустая строка ввода). Вызывает `parsePipeline()` для построения корневого узла AST. После разбора ожидает токен `EOF` (вызывает `expect(TokenType.EOF)`). Возвращает построенный узел `Node` или `error` в случае синтаксической ошибки.
- `- parsePipeline() : (Node, error)` — разбирает цепочку команд, разделённых символом `|`. Сначала вызывает `parseCommand()` для получения левого узла. Пока следующий токен имеет тип `Pipe`, потребляет токен `Pipe`, вызывает `parseCommand()` для правой команды и объединяет результаты в узел `Pipeline` (плоский список команд). Возвращает узел `Node` (либо `Command`, либо `Pipeline`).
- `- parseCommand() : (Command, error)` — разбирает простую команду: сначала в фазе присваиваний считывает токены типа `Word`, пытаясь распознать их как присваивания через `tryParseAssign()`. Для каждого успешно распознанного присваивания добавляет `Assign` в список `Assigns` команды и переходит к следующему токену. Как только встречается слово, не являющееся присваиванием, фаза присваиваний завершается. Далее во второй фазе считывает все оставшиеся токены типа `Word` как аргументы команды, добавляя их `Word` в список `Words`. Возвращает объект `Command` (AST-узел простой команды).
- `- tryParseAssign(w : Word) : (String, Word, Boolean)` — пытается интерпретировать слово `w` как конструкцию присваивания `NAME=value`. Проверяет, что первый сегмент слова является `Unquoted` и `expandable` (присваивание не распознаётся, если слово целиком заключено в кавычки). Проверяет наличие символа `=` в тексте первого сегмента. Извлекает имя переменной до `=` и проверяет его на соответствие правилу идентификатора `[A-Za-z_][A-Za-z0-9_]*`. Если условия выполнены — формирует пару `(name, valueWord, true)`, где `valueWord` представляет значение после `=` (в виде `Word`, без раскрытия). В противном случае возвращает `(String, Word, false)`.
- `- peek() : Token` — возвращает текущий токен по индексу `pos`, не сдвигая указатель чтения. Если `pos` выходит за границы списка — возвращает токен `EOF`.
- `- consume() : Token` — возвращает текущий токен (результат `peek()`), после чего сдвигает `pos` на 1 (переходит к следующему токену).
- `- expect(t : TokenType) : error` — проверяет, что текущий токен имеет тип `t`. Если совпадает — вызывает `consume()` и возвращает `nil`. Если не совпадает — возвращает `error` синтаксического анализа (нарушение ожидаемой структуры).

ParseError

- **Назначение.** Представляет ошибку синтаксического разбора, возникшую на этапе парсинга.

Поля:

- `- msg : String` — текстовое описание ошибки разбора.
- `- pos : Integer` — позиция (номер токена или позиция в исходном вводе), соответствующая месту возникновения ошибки.

Метод:

- `+ Error() : String` — возвращает строковое представление ошибки (содержит `msg` и, при наличии, информацию о позиции).

Node (интерфейс AST)

- **Назначение.** Базовый интерфейс всех узлов абстрактного синтаксического дерева. Определяет замкнутый набор типов узлов AST.
- Стереотип: `{disjoint, complete}` — множество реализаций интерфейса является непересекающимся и полным (запечатанным). Обход узлов AST осуществляется по известным типам реализаций.

Pipeline (узел AST)

- **Назначение.** Представляет пайплайн — цепочку простых команд, соединённых оператором `|`. Реализует интерфейс `Node`.

Поле:

- `- commands : Node [*]` — список узлов команд, входящих в пайплайн. Каждый элемент — узел AST (`Command` или, в общем случае, `Node`), образующий последовательность команд пайплайна (плоская структура).

Command (узел AST — простая команда)

- **Назначение.** Представляет простую команду: набор присваиваний, предшествующих аргументам, и список аргументов (слова), определяющих имя команды и её параметры. Реализует интерфейс `Node`.

Поля:

- `+ Assigns : Assign [*]` — список присваиваний переменных, заданных перед командой (например, `VAR=value cmd`). Присваивания располагаются в начале простой команды и применяются к окружению данной команды.
- `+ Words : Word [*]` — список слов, образующих аргументы простой команды. Первое слово (при наличии) интерпретируется как имя команды, последующие — как аргументы команды. Значения слов представлены в виде `Word` (без предварительного раскрытия переменных).

Assign (присваивание переменной)

- **Назначение.** Представляет отдельное присваивание переменной вида `NAME=value`.

Поля:

- `- name : String` — имя переменной.
- `- value : Word` — значение присваивания, представленное в виде `Word`. Раскрытие переменных в значении выполняется на этапе исполнения (экспандером) в соответствии с политикой раскрытия для присваиваний.

1.4 Executor

Компонент исполнения. Обходит AST, управляет потоком выполнения, создаёт и модифицирует контексты исполнения для команд и пайплайнов, координирует работу экспандера, производит выбор между встроенной и внешней командой.

Executor (реализация)

- **Назначение.** Владелец потока управления исполнением. Обходит узлы AST (`Node`), для каждого типа узла выбирает соответствующую ветку исполнения, вызывает экспандер в нужные моменты с различной политикой раскрытия, осуществляет поиск команд в реестре, формирует каналы для пайплайнов, запускает встроенные и внешние команды.

Поле:

- `- expander : Expander` — экземпляр сервиса раскрытия переменных (реализация интерфейса `Expander`). Используется методами `execSimple()` и `execPipeline()` для раскрытия значений присваиваний и аргументов команд.

Методы:

- `+ Execute(sh : Interpreter, ctx : ExecContext, n : Node) : Integer` — точка входа исполнения узла AST. Выполняет диспетчеризацию по типу узла `n`: если `n` является `Command` (простая команда) — вызывает `execSimple(sh, ctx, n)`. Если `n` является `Pipeline` — вызывает `execPipeline(sh, ctx, n)`. Возвращает код завершения выполненного узла.
- `- execSimple(sh : Interpreter, ctx : ExecContext, c : Command) : Integer` — исполняет простую команду. Алгоритм работы: создаёт локальное окружение `localEnv = ctx.Env.Clone()` (копия окружения контекста). Для каждого присваивания из `c.Assigns` вызывает `expander.ExpandAssignValue(a.Value, ctx.Env)` для раскрытия значения (без разделения по полям), затем устанавливает `localEnv.Set(a.Name, val)`. Если список аргументов команды пуст (`len(c.Words)==0`) — это случай «только присваивания»: для каждого присваивания раскрывает его значение относительно текущего окружения и устанавливает соответствующую переменную в глобальном окружении `sh.env.Set(a.Name, value)`, возвращает `0`. Иначе вызывает `expander.ExpandArgs(c.Words, localEnv)` для раскрытия всех аргументов (с возможным

разделением по пробелам для нецитированных частей), формируя список строковых аргументов `args`. Если `args` пуст — возвращает `0` (раскрытие дало пустое имя команды). Берёт `name = args[0]`, выполняет поиск в реестре команд: `sh.registry.Get(name)`. Если найдена встроенная команда — вызывает `builtin.Run(sh, ctx.WithEnv(localEnv), args[1:])` (передаёт команду без имени, с окружением, ограниченным локальным контекстом команды), возвращает её код возврата. Если команда не найдена в реестре — вызывает `execExternal(ctx.WithEnv(localEnv), args)` для запуска внешней программы, возвращает её код возврата.

- `- execPipeline(sh : Interpreter, ctx : ExecContext, p : Pipeline) : Integer` — исполняет пайплайн из последовательности команд. Создаёт каналы для соединения потоков вывода предыдущей команды с потоком ввода следующей. Запускает команды пайплайна последовательно, для каждой команды формирует `ExecContext` с перенаправленными потоками (`stdin/stdout`) в соответствии с позицией команды в пайплайне (первая — читает из `ctx.Stdin`, последняя — пишет в `ctx.Stdout`, промежуточные — через каналы). Для каждой команды вызывает `Execute(sh, cmdCtx, cmdNode)` (или непосредственно ветку исполнения простой команды). Ожидает завершения всех команд пайплайна. Возвращает код возврата последней команды пайплайна.
- `- execExternal(ctx : ExecContext, args : String [*]) : Integer` — запускает внешнюю программу. Формирует команду запуска по списку `args` (первый элемент — имя исполняемого файла, остальные — аргументы). Подключает потоки: `stdin = ctx.Stdin`, `stdout = ctx.Stdout`, `stderr = ctx.Stderr`. Формирует окружение внешнего процесса из `ctx.Env.Environ()` (список `NAME=VALUE` только для экспортируемых переменных). Выполняет запуск внешнего процесса. При успешном завершении возвращает код возврата процесса. При ошибке: если ошибка представляет собой завершение процесса с ненулевым кодом — возвращает соответствующий код возврата; в иных случаях выводит сообщение об ошибке в `ctx.Stderr` и возвращает `127`.
- `- startExternal(ctx : ExecContext, args : String [*]) : (exec.Cmd, error)` — низкоуровневый запуск внешнего процесса. Создаёт объект команды внешнего процесса по `args`, настраивает потоки ввода-вывода (`Stdin`, `Stdout`, `Stderr`) из `ctx`, устанавливает переменные окружения `Env = ctx.Env.Environ()`, выполняет подготовку к запуску процесса. Возвращает пару `(cmd, error)` (готовый к запуску/запущенный объект процесса и признак ошибки подготовки).

Expander (реализация сервиса раскрытия переменных)

- **Назначение.** Чистый сервис раскрытия подстановок переменных. Выполняет интерполяцию `$VAR` с учётом кавычек, производит (или не производит) разделение по полям в зависимости от контекста вызова (присваивание или аргументы команды). Не хранит состояния между вызовами.

Методы:

- `+ ExpandArgs(words : Word [*], env : Environment) : String [*]` — раскрывает переменные во всех словах списка `words` для использования в качестве аргументов команды. Для каждого `Word` вызывает `ExpandWord(w, env)` (который может вернуть несколько строковых токенов при разделении по полям для нецитированных сегментов), объединяет результаты всех слов в единый список строковых аргументов. Возвращает массив `String [*]` — список раскрытых аргументов команды.
- `+ ExpandWord(w : Word, env : Environment) : String [*]` — раскрывает переменные внутри одного слова `w`. Обрабатывает сегменты `WordPart` с учётом `quoteKind` и флага `expandable`. Для раскрытия значений переменных использует `expandVars()` применительно к содержимому сегментов, где разрешено раскрытие. Выполняет сборку результата с возможным разделением по полям (`splitFields`) для тех частей слова, которые не были защищены кавычками (сегменты `Unquoted`). Возвращает список строковых токенов после раскрытия (может содержать несколько элементов, если произошло полеобразование).
- `+ ExpandAssignValue(w : Word, env : Environment) : String` — раскрывает значение слова `w` для присваивания переменной. Политика раскрытия отличается от аргументов: разделение по полям не выполняется. Обрабатывает сегменты слова, применяет `expandVars()` там, где разрешено раскрытие (`expandable`), и склеивает результат в одну строку без вызова `splitFields()`. Возвращает единственное строковое значение присваивания.
- `- expandVars(s : String, env : Environment) : String` — выполняет замену подстановок переменных (`$VAR`) в строке `s` на значения, получаемые из `env.Get(name)`. Обрабатывает синтаксис подстановки переменных в рамках незащищённых/соответствующих контекстов, подставляет пустую строку, если переменная не определена. Возвращает строку с выполненными подстановками.

- `- expandParts(parts : WordPart [*], env : Environment, split : Boolean) : String [*]` — обрабатывает список сегментов `parts`. Для каждого сегмента, в зависимости от `quoteKind` и `expandable`, применяет раскрытие переменных (`expandVars()`) к текстовому содержимому. Формирует промежуточные строковые фрагменты. Если параметр `split` равен `true` — применяет разделение по пробельным полям (`splitFields()`) к фрагментам, полученным из нецитированных сегментов (`Unquoted`); для сегментов, защищённых кавычками (`Single`, `Double`), разделение не применяется. Возвращает список строковых токенов после обработки.
- `- splitFields(s : String) : String [*]` — разделяет строку `s` на поля по пробельным символам. Игнорирует пустые поля, возникающие при множественных пробелах. Возвращает массив непустых строковых полей.

ExecContext (контекст исполнения одной команды)

- **Назначение.** Описывает то, что видит одна команда в момент исполнения: её потоки ввода-вывода и окружение. Изолирует контекст конкретной команды от глобального состояния, что позволяет корректно обрабатывать пайплайны, локальные присваивания (`VAR=x cmd`) и перенаправления потоков.

Поля:

- `- stdin : Reader` — поток ввода, доступный команде. Для команды верхнего уровня соответствует потокам REPL, для команды внутри пайплайна — концу канала (pipe reader), соединяющего с предыдущей командой.
- `- stdout : Writer` — поток вывода команды. Для последней команды пайплайна — выходной поток контекста, для промежуточных — начало канала (pipe writer), соединяющего со следующей командой.
- `- stderr : Writer` — поток вывода ошибок команды. Как правило, общий для всех команд в рамках текущего контекста исполнения (не перенаправляется пайплайном).
- `- env : Environment` — окружение, видимое команде. Может быть копией глобального окружения с локальными изменениями (например, с добавленными переменными из присваиваний перед командой). Для встроенных команд используется именно это окружение; для внешних команд из него формируется список экспортируемых переменных (`Environ()`).

Методы:

- `+ WithStreams(in : Reader, out : Writer, err : Writer) : ExecContext` — создаёт новый экземпляр `ExecContext` на основе текущего, заменяя потоки ввода-вывода на переданные `in`, `out`, `err`. Исходный контекст не изменяется (иммутабельное обновление). Возвращает новый `ExecContext` с обновлёнными потоками.
- `+ WithEnv(env : Environment) : ExecContext` — создаёт новый экземпляр `ExecContext` на основе текущего, заменяя окружение на `env`. Исходный контекст не изменяется. Возвращает новый `ExecContext` с обновлённым окружением.
- `+ Clone() : ExecContext` — создаёт копию текущего контекста (поверхностное/логическое копирование полей: потоки копируются по ссылке на абстракции `Reader/Writer`, `env` копируется как ссылка на соответствующий экземпляр `Environment`, используемый в данном контексте). Возвращает независимую копию `ExecContext` с теми же значениями полей.

1.5 Models

Общие типы данных, используемые несколькими компонентами. Представляют собой конкретные структуры данных (DTO), не инкапсулируемые интерфейсами.

Word

- **Назначение.** Представляет слово в виде последовательности сегментов с сохранением информации о кавычках. Позволяет отделить этап лексического разбиения (сохранение кавычковой структуры) от этапа раскрытия переменных (экспансия).

Поле:

- `- parts : WordPart [*]` — список сегментов слова. Каждый сегмент описывает фрагмент текста с его типом кавычек и возможностью раскрытия переменных.

Методы:

- `+ IsEmpty() : Boolean` — проверяет, пусто ли слово. Возвращает `true`, если список `parts` пуст или все сегменты не содержат значимого текста. Иначе — `false`.
- `+ IsFullyQuoted() : Boolean` — проверяет, полностью ли слово защищено кавычками. Возвращает `true`, если слово состоит только из сегментов с `quoteKind = Single` или `quoteKind = Double` (не содержит нецитированных сегментов `Unquoted`). Иначе — `false`.

WordPart (тип данных)

- **Назначение.** Описывает один сегмент слова: фрагмент текста и способ его интерпретации при раскрытии переменных (защищён ли кавычками, разрешено ли раскрытие `$`).

Поля:

- `- text : String` — текстовое содержимое сегмента (после удаления внешних кавычек, определяющих данный сегмент).
- `- quoteKind : QuoteKind` — вид кавычек, определяющий данный сегмент: `Unquoted`, `Single` или `Double`.
- `- expandable : Boolean` — признак разрешения раскрытия переменных `$VAR` внутри данного сегмента. Значение зависит от `quoteKind`: для `Single` — `false`, для `Double` и `Unquoted` — `true` (в рамках правил лексического анализа).

QuoteKind (перечисление)

- **Назначение.** Определяет тип кавычкового оформления сегмента слова.
- `Unquoted` — нецитированный сегмент. Допускает раскрытие переменных и разделение по полям при раскрытии аргументов.
- `Single` — сегмент, заключённый в одинарные кавычки. Раскрытие переменных не выполняется, разделение по полям не выполняется.
- `Double` — сегмент, заключённый в двойные кавычки. Раскрытие переменных разрешено, разделение по полям не выполняется.

1.6 Builtins

Компонент встроенных команд интерпретатора. Содержит интерфейс команды и реализации стандартных встроенных команд.

Command (интерфейс команды)

- **Назначение.** Единый интерфейс для всех команд (встроенных и пользовательских). Определяет контракт исполнения команды: имя команды и метод её запуска.

Методы:

- `+ Name() : String` — возвращает имя команды, под которым она регистрируется в `Registry`. Используется при поиске команды по имени.
- `+ Run(sh : Interpreter, ctx : ExecContext, args : String [*]) : Integer` — выполняет логику команды. Параметры: `sh` — ссылка на интерпретатор (предоставляет доступ к глобальному состоянию: окружению, флагам выхода и т.д.; используется командами, изменяющими глобальное состояние), `ctx` — контекст исполнения команды (потоки ввода-вывода и окружение, видимые команде), `args` — список раскрытых строковых аргументов команды без имени команды. Возвращает код возврата команды (`Integer`).

CatCommand

- **Назначение.** Встроенная команда `cat`. Выводит содержимое указанных файлов в `ctx.Stdout`. Если аргументы не заданы — копирует данные из `ctx.Stdin` в `ctx.Stdout` (поддержка работы в пайплайне).
- Реализует интерфейс `Command`.
- `+ Name() : String` — возвращает имя `"cat"`.
- `+ Run(sh : Interpreter, ctx : ExecContext, args : String [*]) : Integer` — выполняет логику команды `cat`. При отсутствии аргументов выполняет побайтовое копирование содержимого `ctx.Stdin` в `ctx.Stdout`. При наличии аргументов последовательно выводит содержимое каждого указанного файла в `ctx.Stdout`. При ошибке открытия/чтения файла выводит сообщение в `ctx.Stderr` и продолжает обработку

следующих файлов (либо завершает с ненулевым кодом возврата в соответствии с семантикой команды). Возвращает 0 при успешном выполнении, иначе ненулевой код возврата.

EchoCommand

- **Назначение.** Встроенная команда echo. Выводит аргументы через пробел в ctx.Stdout, завершает вывод символом перевода строки (по умолчанию).
- Реализует интерфейс Command.
- + Name() : String — возвращает имя "echo".
- + Run(sh : Interpreter, ctx : ExecContext, args : String [*]) : Integer — выводит элементы args, разделённые пробелом, в ctx.Stdout. Если аргументы отсутствуют — выводит только символ перевода строки. Возвращает 0.

WcCommand

- **Назначение.** Встроенная команда wc. Подсчитывает количество строк, слов и байтов. Если аргументы заданы — обрабатывает указанные файлы. Если аргументы не заданы — читает данные из ctx.Stdin (поддержка работы в пайплайне).
- Реализует интерфейс Command.
- + Name() : String — возвращает имя "wc".
- + Run(sh : Interpreter, ctx : ExecContext, args : String [*]) : Integer — выполняет подсчёт статистики. При отсутствии аргументов читает данные из ctx.Stdin, вычисляет количество строк, слов, байтов и выводит результат в ctx.Stdout. При наличии аргументов обрабатывает каждый файл по отдельности, выводит статистику для каждого файла и при необходимости итоговую сумму. При ошибке доступа к файлу выводит сообщение в ctx.Stderr. Возвращает 0 при успешном выполнении, иначе ненулевой код возврата.

PwdCommand

- **Назначение.** Встроенная команда pwd. Выводит текущую рабочую директорию в ctx.Stdout.
- Реализует интерфейс Command.
- + Name() : String — возвращает имя "pwd".
- + Run(sh : Interpreter, ctx : ExecContext, args : String [*]) : Integer — определяет текущую рабочую директорию (через системное обращение к рабочему каталогу), выводит её в ctx.Stdout. При ошибке получения директории выводит сообщение в ctx.Stderr и возвращает ненулевой код возврата. При успешном выполнении возвращает 0.

ExitCommand

- **Назначение.** Встроенная команда exit. Инициирует завершение работы интерпретатора. При наличии аргумента использует его как код завершения, при отсутствии — использует код возврата последней команды.
- Реализует интерфейс Command.
- + Name() : String — возвращает имя "exit".
- + Run(sh : Interpreter, ctx : ExecContext, args : String [*]) : Integer — обрабатывает запрос на выход. Устанавливает флаг sh.shouldExit = true. Если аргумент передан — пытается преобразовать его в целочисленный код завершения и устанавливает sh.exitCode в полученное значение. Если аргумент не передан — устанавливает sh.exitCode в значение sh.lastStatus (код возврата последней выполненной команды). Возвращает 0 (код возврата самой команды exit перед завершением работы).

2. Взаимодействие компонентов (описание внутри методов)

Взаимодействие между компонентами осуществляется через вызовы методов, без явного перечисления связей между контейнерами. Ниже кратко описано, как компоненты координируют работу в ключевых точках.

- **Инициализация и запуск (App → Repl).** App.main() запускает цикл Repl.Run(). Repl владеет экземпляром Interpreter и использует его для выполнения строк (interpr.ExecuteLine()) и проверки условия выхода (interpr.ShouldExit()).
- **Обработка строки (Repl → Interpreter).** Repl.Run() читает строку через readline() (используя stdin/stdout) и передаёт её в Interpreter.ExecuteLine(line). Interpreter.ExecuteLine()

последовательно вызывает lexer.Tokenize() (преобразование строки в токены), parser.Parse() (построение AST). При успешном разборе создаёт контекст через defaultContext() и передаёт AST в executor.Execute() для исполнения. Возвращённый код возврата сохраняется в lastStatus.

- **Построение структуры (Lexer → Parser).** Lexer.Tokenize() формирует список объектов Token, каждый из которых содержит Word, собранный из WordPart с типами QuoteKind. Parser последовательно читает токены (peek(), consume()), распознаёт присваивания через tryParseAssign() и строит узлы AST (Command, Pipeline, Assign), реализующие интерфейс Node.
- **Исполнение (Interpreter → Executor → Expander/Registry).** Executor.Execute() диспетчеризует узел AST: для простой команды вызывает execSimple(), для пайплайна — execPipeline(). В execSimple() Executor использует expander.ExpandAssignValue() (для присваиваний, без разделения по полям) и expander.ExpandArgs() (для аргументов, с возможным разделением по полям). После раскрытия аргументов выполняет поиск команды в Registry.Get(name): при нахождении вызывает Command.Run(sh, ctx.WithEnv(...), args[1:]); при отсутствии — вызывает execExternal() для запуска внешней программы, используя Environment.Environ() для формирования окружения процесса. В execPipeline() Executor формирует цепочку контекстов с перенаправленными потоками (WithStreams) и последовательно исполняет команды пайплайна.
- **Контекст и окружение (Executor ↔ ExecContext ↔ Environment).** ExecContext инкапсулирует stdin/stdout/stderr (типы Reader/Writer) и env (Environment). Методы WithEnv, WithStreams, Clone() создают новые контексты без изменения исходных. Environment.Clone() используется для получения локального окружения команды, Environment.Environ() — только при запуске внешнего процесса. Environment.FromOS() формирует начальное глобальное окружение при инициализации.
- **Границы с внешним миром.** Потоки ввода-вывода абстрагированы интерфейсами Reader/Writer. Запуск внешних программ осуществляется через Executor.execExternal()/startExternal() (с использованием потоков и окружения из ExecContext). Импорт системных переменных окружения выполняется методом Environment.FromOS().

3. Жизненный цикл обработки команды

Пример: FILE=test.txt cat $FILE | wc

1. **Чтение (REPL).** Repl.readline() читает строку ввода и передаёт её в Interpreter.ExecuteLine().
2. **Лексический анализ.** Lexer.Tokenize() формирует поток токенов: [FILE=test.txt, cat, $FILE, |, wc]. Слова представлены в виде Word из соответствующих WordPart (учитывая отсутствие/наличие кавычек).
3. **Синтаксический разбор.** Parser.Parse() строит AST: Pipeline, содержащий две команды (Command). Первая команда содержит присваивание Assign(name="FILE", value=Word($FILE/test.txt)) и аргументы [Word(cat), Word($FILE)]. Вторая — аргументы [Word(wc)].
4. **Исполнение пайплайна (Executor).** Executor.Execute() обходит AST: создаёт каналы связи между командами. Для первой команды создаёт локальное окружение (ctx.Env.Clone()), для присваивания FILE вызывает Expander.ExpandAssignValue() (без разделения по полям) и устанавливает FILE=test.txt в локальном окружении. Вызывает Expander.ExpandArgs() для аргументов $FILE → получает ["test.txt"]. Выполняет поиск команды cat в Registry: при обнаружении встроенной команды вызывает CatCommand.Run() с контекстом, у которого env заменён на локальное окружение, а stdout направлен в канал пайплайна. Для второй команды формирует контекст с stdin, подключённым к каналу, stdout — в выходной поток, вызывает WcCommand.Run() (или внешнюю команду при отсутствии встроенной) с соответствующим контекстом.
5. **Завершение.** Executor возвращает код возврата последней команды пайплайна. Interpreter.ExecuteLine() сохраняет его в lastStatus. Управление возвращается в Repl.Run(), который проверяет Interpreter.ShouldExit() и при необходимости завершает цикл.